LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3

Single Linked List

Oleh:

Ahmad Riyadhi **NIM. 2510817110004**

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma & Struktur Data Modul 3 : Single Linked List ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Ahmad Riyadhi
NIM : 2510817110004

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Fadhil Syahdama Mahatma Putra	Andreyan Rizky Baskara, S.Kom., M.Kom
NIM. 2410817210026	NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR TABEL iii
DAFTAR GAMBAR iv
SOAL 1 1
Pembahasan 5
SOAL 2 29
Pembahasan 29
SOAL 3 30
Pembahasan 30
SOAL 4 31
Pembahasan 31
SOAL 5 32
Pembahasan 32
SOAL 6 33
Pembahasan 33
SOAL 7 34
Pembahasan 34
SOAL 8 36
Pembahasan 36
SOAL 9 38
Pembahasan 38
SOAL 10 39
Pembahasan 39
TAUTAN GIT HUB 40

DAFTAR TABEL

Table 1 Source Code Linked List 5

DAFTAR GAMBAR

Gambar 1 Tampilan Awal Program 15
Gambar 2 Tampilan Tambah Data dari Depan 15
Gambar 3 Tampilan Data Setelah Dilakukan Tambah Data Depan 16
Gambar 4 Tampilan Tambah Data dari Belakang 16
Gambar 5 Tampilan Data Setelah Dilakukan Tambah Data Belakang 17
Gambar 6 Tampilan Cari Data Yang Tidak Ada 17
Gambar 7 Tampilan Cari Data Yang Ada 18
Gambar 8 Tampilan Hapus Data Belakang 18
Gambar 9 Tampilan Data Setelah Dilakukan Hapus Data Belakang 19
Gambar 10 Tampilan Hapus Setiap Data Tertentu 19
Gambar 11 Tampilan Data Setelah Dilakukan Hapus Setiap Data Tertentu 20
Gambar 12 Tampilan Hapus Data Depan 20
Gambar 13 Tampilan Data Setelah Dilakukan Hapus Data Depan 21
Gambar 14 Tampilan Sisipkan Node Sebelum Data Tertentu 21
Gambar 15 Tampilan Data Setelah Dilakukan Sisipkan Node Sebelum 22
Gambar 16 Tampilan Sisipkan Node Setelah Data Tertentu 22
Gambar 17 Tampilan Data Setelah Dilakukan Sisipkan Node Setelah 23
Gambar 18 Tampilan Hapus Semua Elemen 23
Gambar 19 Tampilan Data Setelah Dilakukan Hapus Semua Elemen 24
Gambar 20 Tampilan Setelah Selesai Menyelesaiakan Program 24
Gambar 21 Tampilan Nilai yang Diinput dari Depan 29
Gambar 22 Tampilan Nilai yang Diinput dari Belakang 30
Gambar 23 Pencarian Nilai 2 pada Linked List 31
Gambar 24 Pencarian Nilai 7 pada Linked List 32
Gambar 25 Menghapus Data dari Belakang Linked List 33
Gambar 26 Tampilan Linked List Setelah Dihapus 33
Gambar 27 Menghapus Setiap Nilai 3 yang Ada di Linked List 34
Gambar 28 Tampilan Linked List Setelah Setiap Nilai 3 Dihapus 35
Gambar 29 Tampilan Isi dari Linked List 36

SOAL 1

Cobalah program berikut, running, simpan program, dan screenshoot hasil running !

```cpp
#include <conio.h>
#include <iostream>
#include <stdlib.h>

using namespace std;

typedef struct TNode {
    string data;
    TNode *next;
};

TNode *head, *tail;

int pil;
char pilihan[2];
string dataBaru, data;

void init();
int isEmpty();

void tambahDepan();
void tambahBelakang();
void hapusDepan();
void hapusBelakang();
void tampilkan();
void reset();
void cariData();
void hapusData();
void sisipkanSebelum();
void sisipkanSetelah();

int main()
{
    do {
        cout<<"Single Linked List Circular (SLLC)"<<endl;
        cout<<"================================"<<endl;
        cout<<"1. Tambah Depan"<<endl;
        cout<<"2. Tambah Belakang"<<endl;
        cout<<"3. Hapus Depan"<<endl;
        cout<<"4. Hapus Belakang"<<endl;
        cout<<"5. Tampilkan Data"<<endl;
        cout<<"6. Hapus Semua Elemen"<<endl;
        cout<<"7. Cari Data"<<endl;
        cout<<"8. Hapus Setiap Data Tertentu"<<endl;
        cout<<"9. Sisipkan Node/Data Baru Sebelum Data Tertentu"<<endl;
        cout<<"10. Sisipkan Node/Data Baru Setelah Data Tertentu"<<endl;
        cout<<"11. Quit"<<endl;
        cout<<"Pilihan : ";
```

```cpp
        cin>>pilihan;
        pil=atoi(pilihan);

        switch(pil) {
        case 1:
            tambahDepan();
            break;
        case 2:
            tambahBelakang();
            break;
        case 3:
            hapusDepan();
            cout<<"Data \""<<data<<"\" yang berada di depan telah berhasil dihapus."<<endl;
            break;
        case 4:
            hapusBelakang();
            cout<<"Data \""<<data<<"\" yang berada di belakang telah berhasil dihapus."<<endl;
            break;
        case 5:
            tampilkan();
            break;
        case 6:
            reset();
            break;
        case 7:
            cariData();
            break;
        case 8:
            hapusData();
            break;
        case 9:
            sisipkanSebelum();
            break;
        case 10:
            sisipkanSetelah();
            break;
        default:
            cout<<"\nTERIMA KASIH"<<endl;
            cout<<"Program was made by Nama (NIM)."<<endl;
        }

        cout<<"\nPress any key to continue!"<<endl;
        getch();
        system("cls");

    } while (pil<11);
```

```
}

    void init(){
        head = NULL;
        tail = NULL;
    }

    int isEmpty() {
        if(head == NULL) return 1;
        else return 0;
    }

    void tambahDepan() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data = dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            baru->next = head;
            head = baru;
            tail->next = head;
        }
        cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan."<<endl;
    }

    void tambahBelakang() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data = dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            tail->next = baru;
            tail = baru;
```

```
            tail->next = head;
        }
        cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian belakang."<<endl;
    }

    void hapusDepan() {
        if(isEmpty() == 0) {
            TNode *hapus;
            hapus = head;
            data = hapus->data;

            if(head != tail) {
                head = head->next;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void hapusBelakang() {
        if(isEmpty() == 0) {
            TNode *hapus, *newTail;
            hapus = tail;
            data = hapus->data;

            if(head != tail) {
                newTail = head;
                while(newTail->next != tail) {
                    newTail = newTail->next;
                }
                tail = newTail;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void tampilkan() {
        if(isEmpty() == 0) {
            TNode *bantu;
```

```
            bantu = head;

            do {
                cout<<bantu->data<<' ';
                bantu = bantu->next;
            } while(bantu != head);
            cout<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void reset() {
        if(isEmpty() == 0) {
            TNode *bantu, *hapus;
            bantu = head;

            do {
                hapus = bantu;
                bantu = bantu->next;
                delete hapus;
            } while(bantu != head);

            init();
            cout<<"Seluruh elemen pada Linked List telah dibersihkan."<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void cariData() {
        if(isEmpty() == 0) {
            string cari;
            cout<<"Masukkan data yang ingin dicari : ";
            cin>>cari;

            TNode *bantu, *hapus, *newTail, *bantuTampilkan;
            bool apaDitemukan = false;

            bantu = head;

            do {
                if(cari == bantu->data){
                    cout<<"Setiap data yang berada di dalam tanda kurung siku ([...]) "
                        <<"merupakan data yang anda cari."<<endl;
                    cout<<"Linked List : ";
                    bantuTampilkan = head;

                    do {
                        if(cari == bantuTampilkan->data)
```

```
                            cout<<"["<<bantuTampilkan->data<<"] ";
                        else
                            cout<<bantuTampilkan->data<<' ';
                        bantuTampilkan = bantuTampilkan->next;
                    } while(bantuTampilkan != head);

                    apaDitemukan = true;
                    cout<<endl;
                    break;
                }
                bantu = bantu->next;
            } while(bantu != head);

            if(apaDitemukan == false)
                cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void hapusData() {
        if(isEmpty() == 0) {
            string cari;
            cout<<"Masukkan data yang ingin dihapus : ";
            cin>>cari;

            TNode *bantu, *sebelum, *hapus[255], *bantu2;
            int hitung = 0;
            bool apaDitemukan = false;

            bantu = head;

            do {
                bantu2 = bantu;
                if(cari == bantu->data){
                    hapus[hitung++] = bantu;
                    apaDitemukan = true;
                    if(bantu != head && bantu != tail) {
                        sebelum->next = bantu->next;
                        bantu2 = sebelum;
                    }
                }
                sebelum = bantu2;
                bantu = bantu->next;
            } while(bantu != head);

            if(apaDitemukan == true) {
                for(int i = 0; i < hitung; i++) {
```

```
                    if(hapus[i] == head){
                        hapusDepan();
                    } else if(hapus[i] == tail) {
                        hapusBelakang();
                    } else {
                        delete hapus[i];
                    }
                }

                cout<<"Setiap data \""<<cari<<"\" yang terdapat pada Linked List telah dihapus";
            } else cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;

        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void sisipkanSebelum() {
        if(isEmpty() == 0) {
            TNode *bantu, *sebelum;
            string nextData;
            bool apaAda;

            bantu = head;
            sebelum = tail;

            cout<<"Sisipkan data baru sebelum data : ";
            cin>>nextData;

            do {
                if(nextData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    sebelum = bantu;
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
```

```
                baru->next = bantu;

                sebelum->next = baru;

                if(bantu == head){
                    head = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan sebelum data \""<<nextData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<nextData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void sisipkanSetelah() {
        if(isEmpty() == 0) {
            TNode *bantu;
            string prevData;
            bool apaAda;

            bantu = head;

            cout<<"Sisipkan data baru setelah data : ";
            cin>>prevData;

            do {
                if(prevData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
                baru->next = bantu->next;
```

```
                bantu->next = baru;

                if(bantu == tail){
                    tail = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan setalah data \""<<prevData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<prevData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

```

Pembahasan

Table 1 Source Code Linked List

```
#include <conio.h>
#include <iostream>
#include <stdlib.h>

using namespace std;

typedef struct TNode{
    string data;
    TNode *next;
};

TNode *head, *tail;

int pil;
char pilihan[2];
string dataBaru, dataDelete;

void init();
int isEmpty();

void tambahDepan();
void tambahBelakang();
void hapusDepan();
void hapusBelakang();
void tampilkan();
void reset();
void cariData();
void hapusData();
void sisipkanSebelum();
void sisipkanSetelah();

int main()
{
    do {
```

```
        cout<<"Single Linked List Circular (SLLC)"<<endl;
        cout<<"==================================="<<endl;
        cout<<"1. Tambah Depan"<<endl;
        cout<<"2. Tambah Belakang"<<endl;
        cout<<"3. Hapus Depan"<<endl;
        cout<<"4. Hapus Belakang"<<endl;
        cout<<"5. Tampilkan Data"<<endl;
        cout<<"6. Hapus Semua Elemen"<<endl;
        cout<<"7. Cari Data"<<endl;
        cout<<"8. Hapus Setiap Data Tertentu"<<endl;
        cout<<"9. Sisipkan Node/Data Baru Sebelum Data Tertentu"<<endl;
        cout<<"10. Sisipkan Node/Data Baru Setelah Data Tertentu"<<endl;
        cout<<"11. Quit"<<endl;
        cout<<"Pilihan : ";
        cin>>pilihan;
        pil=atoi(pilihan);

        switch(pil) {
        case 1:
            tambahDepan();
            break;
        case 2:
            tambahBelakang();
            break;
        case 3:
            hapusDepan();
            cout<<"Data \""<<dataDelete<<"\" yang berada di depan telah berhasil dihapus."<<endl;
            break;
        case 4:
            hapusBelakang();
            cout<<"Data \""<<dataDelete<<"\" yang berada di belakang telah berhasil dihapus."<<endl;
            break;
        case 5:
            tampilkan();
            break;
        case 6:
            reset();
            break;
        case 7:
            cariData();
            break;
```

```
        case 8:
            hapusData();
            break;
        case 9:
            sisipkanSebelum();
            break;
        case 10:
            sisipkanSetelah();
            break;
        default:
            cout<<"\nTERIMA KASIH"<<endl;
            cout<<"Program was made by Ahmad Riyadhi (2510817110004)."<<endl;
        }

        cout<<"\nPress any key to continue!"<<endl;
        getch();
        system("cls");

    } while (pil < 11);
}

    void init() {
        head = NULL;
        tail = NULL;
    }

    int isEmpty() {
        if(head == NULL) return 1;
        else return 0;
    }

    void tambahDepan() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data = dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            baru->next = head;
```

```
            head = baru;
            tail->next = head;
        }
        cout<< "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan."<<endl;
    }

    void tambahBelakang() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data =  dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            tail->next = baru;
            tail = baru;
            tail->next = head;
        }
        cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan dibagian belakang."<<endl;
    }

    void hapusDepan() {
        if(isEmpty() == 0) {
            TNode *hapus;
            hapus = head;
            dataDelete = hapus->data;

            if(head != tail) {
                head = head->next;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

```

```
    void hapusBelakang(){
        if(isEmpty() == 0) {
            TNode *hapus, *newTail;
            hapus = tail;
            dataDelete = hapus->data;

            if(head != tail) {
                newTail = head;
                while(newTail->next != tail) {
                    newTail = newTail->next;
                }
                tail = newTail;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void tampilkan() {
        if(isEmpty() == 0) {
            TNode *bantu;
            bantu = head;

            do {
                cout<<bantu->data<<' ';
                bantu = bantu->next;
            } while(bantu != head);
            cout<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void reset() {
        if(isEmpty() == 0) {
            TNode *bantu, *hapus;
            bantu = head;

            do {
                hapus = bantu;
                bantu = bantu->next;
                delete hapus;
```

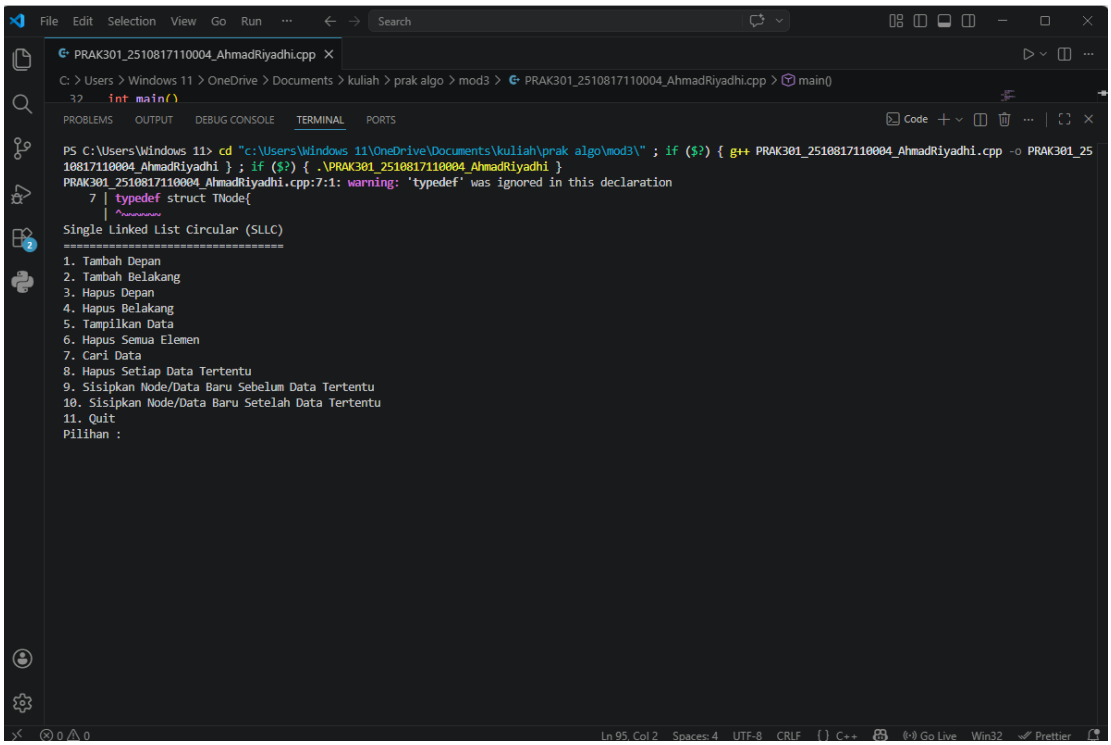
```
            } while(bantu != head);

            init();
            cout<<"Seluruh elemen pada Linked List telah dibersihkan."<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void cariData(){
        if(isEmpty() == 0) {
            string cari;
            cout<<"Masukan data yang ingin dicari : ";
            cin>>cari;

            TNode *bantu, *hapus, *newTail, *bantuTampilkan;
            bool apaDitemukan = false;

            bantu = head;

            do {
                if(cari == bantu->data){
                    cout<<"Setiap data yang berada di dalam tanda kurung siku ([...]) "
                        <<"merupakan data yang anda cari."<<endl;
                    cout<<"Linked List : ";
                    bantuTampilkan = head;

                    do {
                        if(cari == bantuTampilkan->data)
                            cout<<"["<<bantuTampilkan->data<<"] ";
                        else
                            cout<<bantuTampilkan->data<<' ';
                        bantuTampilkan = bantuTampilkan->next;
                    } while(bantuTampilkan != head);

                    apaDitemukan = true;
                    cout<<endl;
                    break;
                }
```

```
                bantu = bantu->next;
            } while(bantu != head);

            if(apaDitemukan == false)
                cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void hapusData(){
        if(isEmpty() == 0) {
            string cari;
            cout<<"Masukkan data yang ingin dihapus : ";
            cin>>cari;

            TNode *bantu, *sebelum, *hapus[225], *bantu2;
            int hitung = 0;
            bool apaDitemukan = false;

            bantu = head;

            do {
                bantu2 = bantu;
                if(cari == bantu->data){
                    hapus[hitung++] = bantu;
                    apaDitemukan = true;
                    if(bantu != head && bantu != tail) {
                        sebelum->next = bantu->next;
                        bantu2 = sebelum;
                    }
                }
                sebelum = bantu2;
                bantu = bantu->next;
            } while(bantu != head);

            if(apaDitemukan == true) {
                for(int i = 0; i < hitung; i++) {
                    if(hapus[i] == head){
                        hapusDepan();
                    } else if(hapus[i] == tail) {
                        hapusBelakang();
                    } else {
                        delete hapus[i];
                    }
```

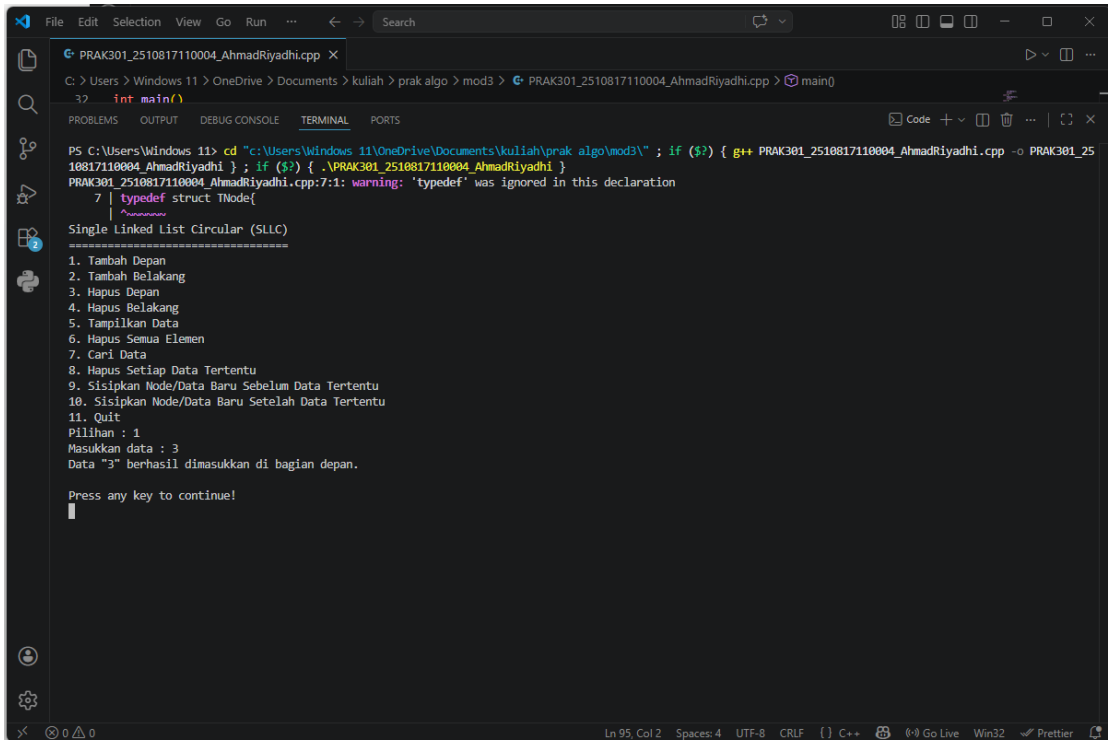
```
                }

                cout<<"Setiap data \""<<cari<<"\" yang terdapat pada Linked List telah dihapus";
            } else cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;

        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void sisipkanSebelum() {
        if(isEmpty() == 0) {
            TNode *bantu, *sebelum;
            string nextData;
            bool apaAda;

            bantu = head;
            sebelum = tail;

            cout<<"Sisipkan data baru sebelum data : ";
            cin>>nextData;

            do {
                if(nextData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    sebelum = bantu;
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
                baru->next = bantu;

                sebelum->next = baru;
```

```

                if(bantu == head){
                    head = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan sebelum data \""<<nextData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<nextData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak tedapat data pada Linked List."<<endl;
    }

    void sisipkanSetelah() {;
        if(isEmpty() == 0) {
            TNode *bantu;
            string prevData;
            bool apaAda;

            bantu = head;

            cout<<"Sisipkan data baru setelah data : ";
            cin>>prevData;

            do {
                if(prevData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
                baru->next = bantu->next;
```

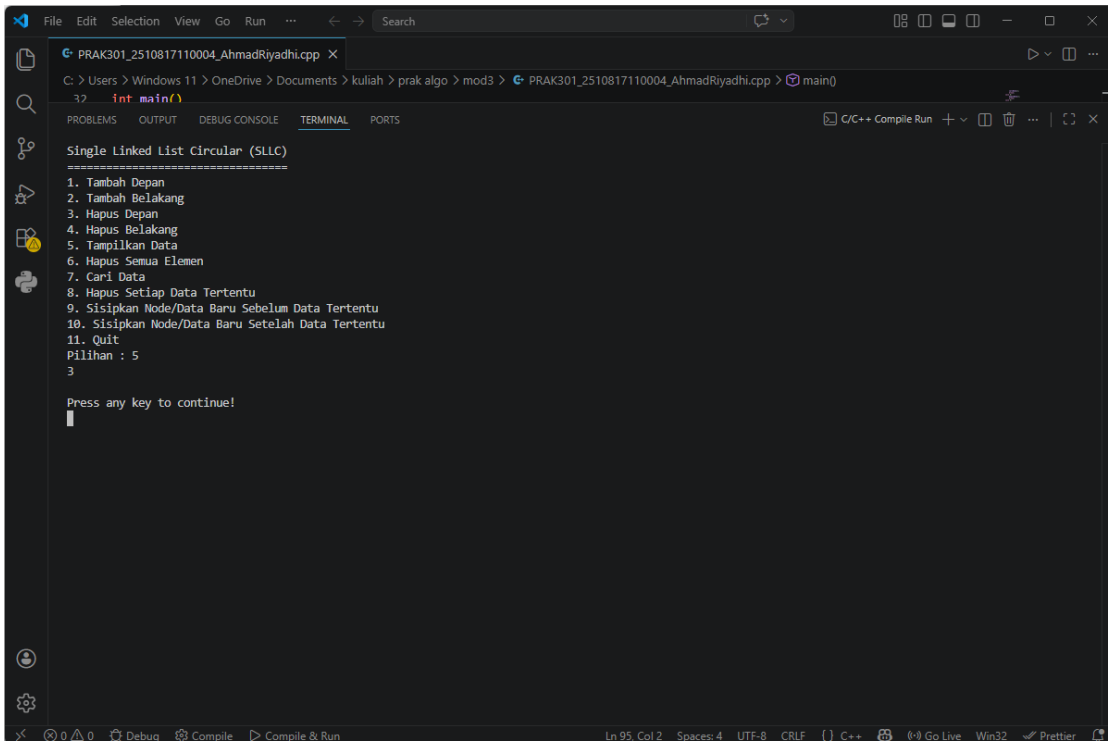
```

                bantu->next = baru;

                if(bantu == tail){
                    tail = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan setelah data \""<<prevData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<prevData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }
```



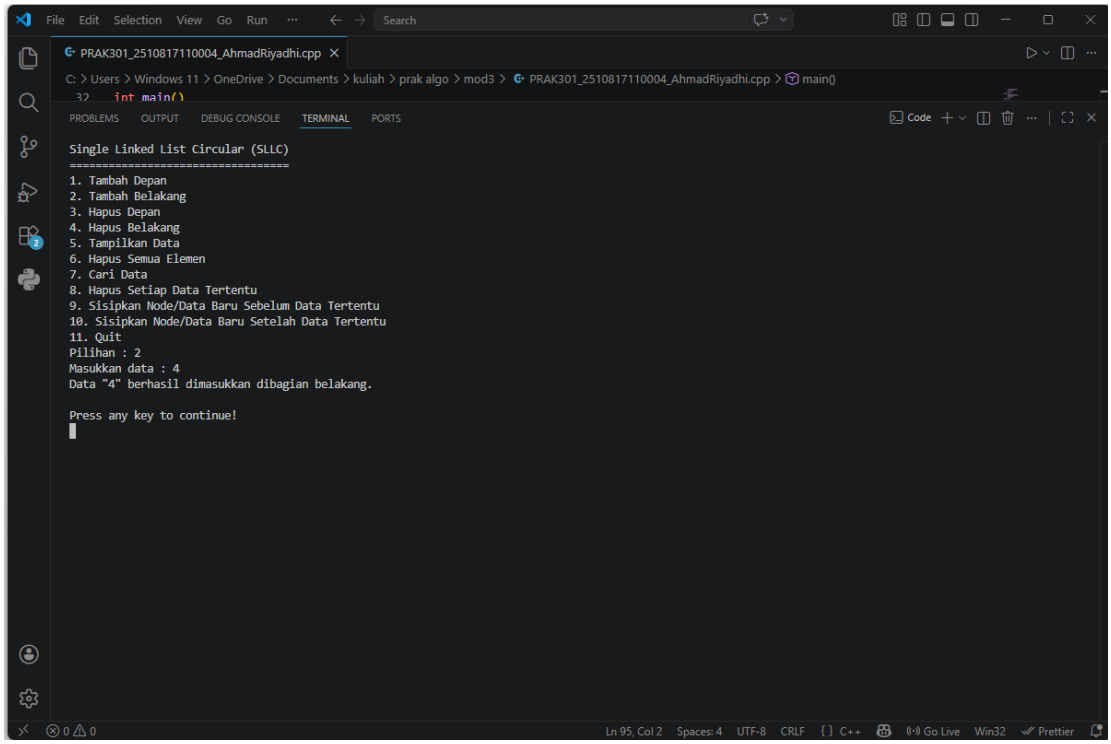
Gambar 1 Tampilan Awal Program



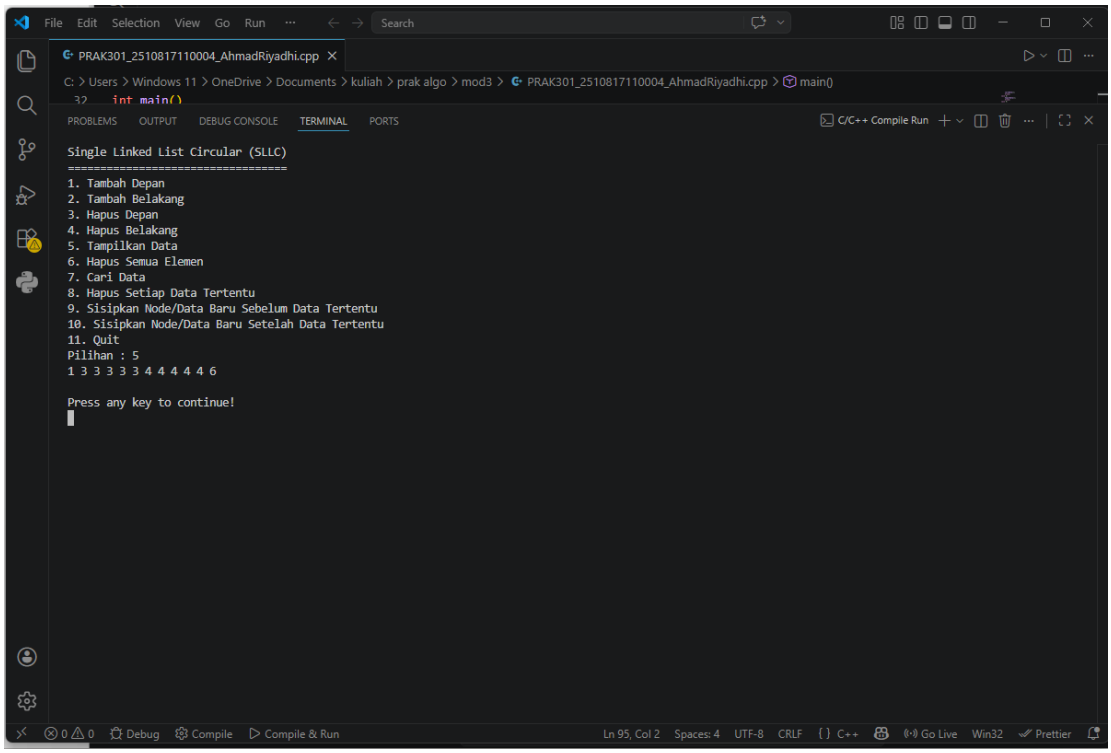
Gambar 2 Tampilan Tambah Data dari Depan



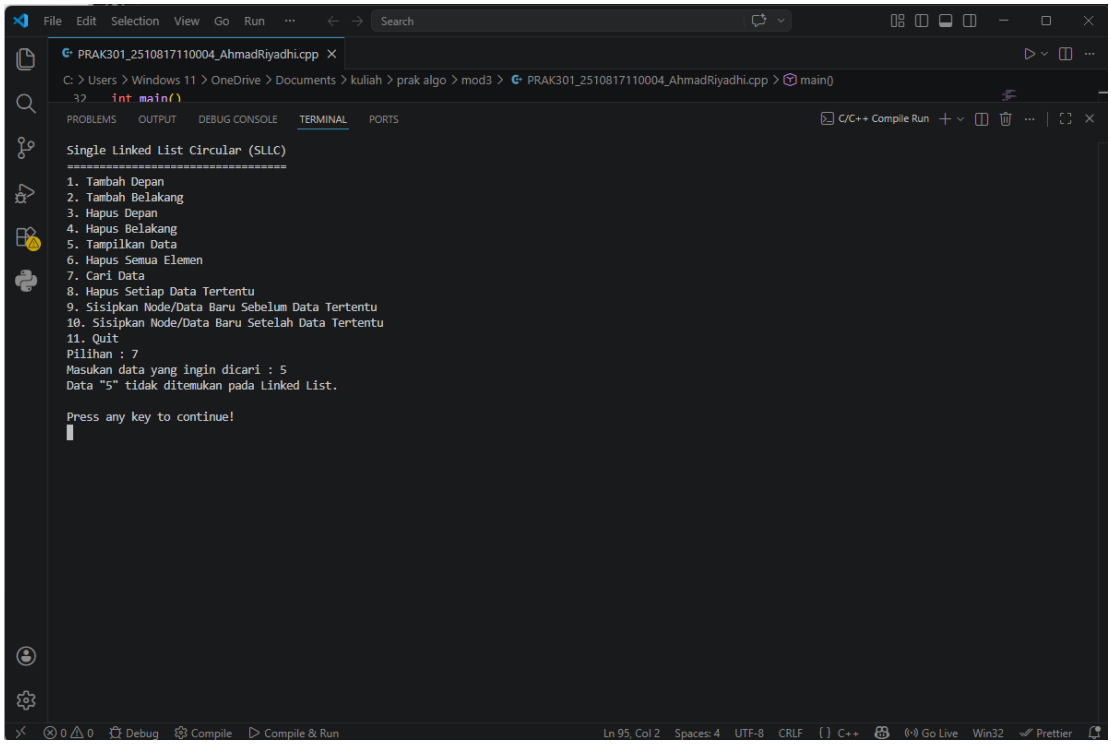
Gambar 3 Tampilan Data Setelah Dilakukan Tambah Data Depan



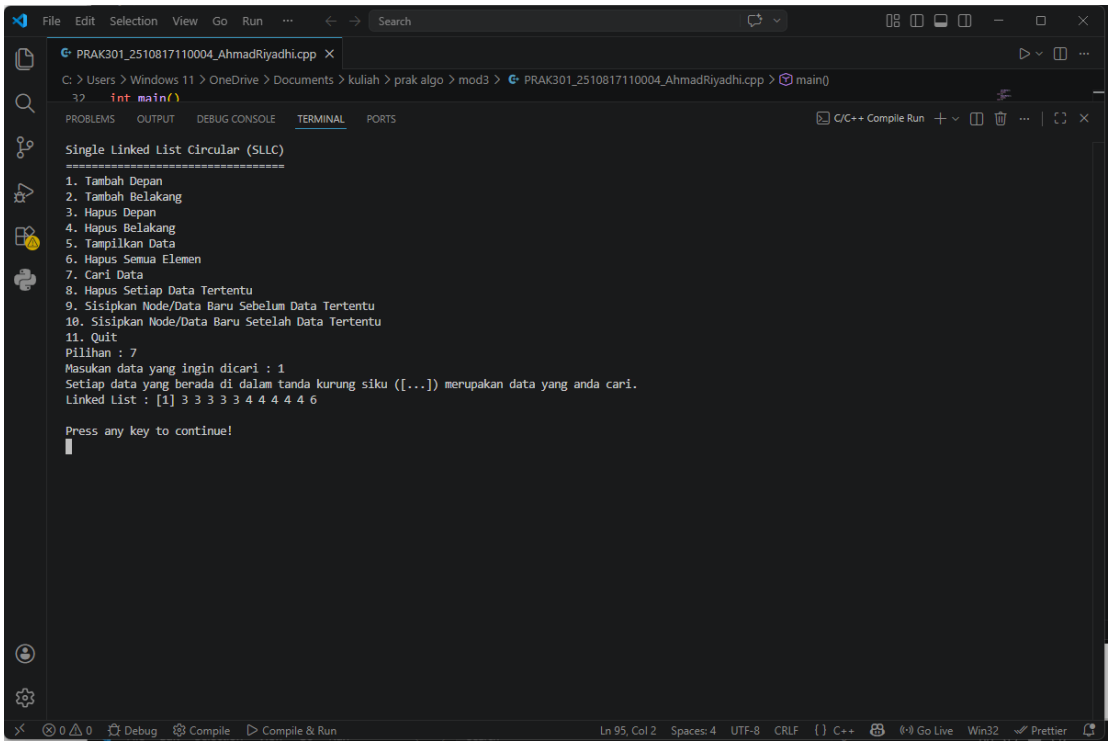
Gambar 4 Tampilan Tambah Data dari Belakang



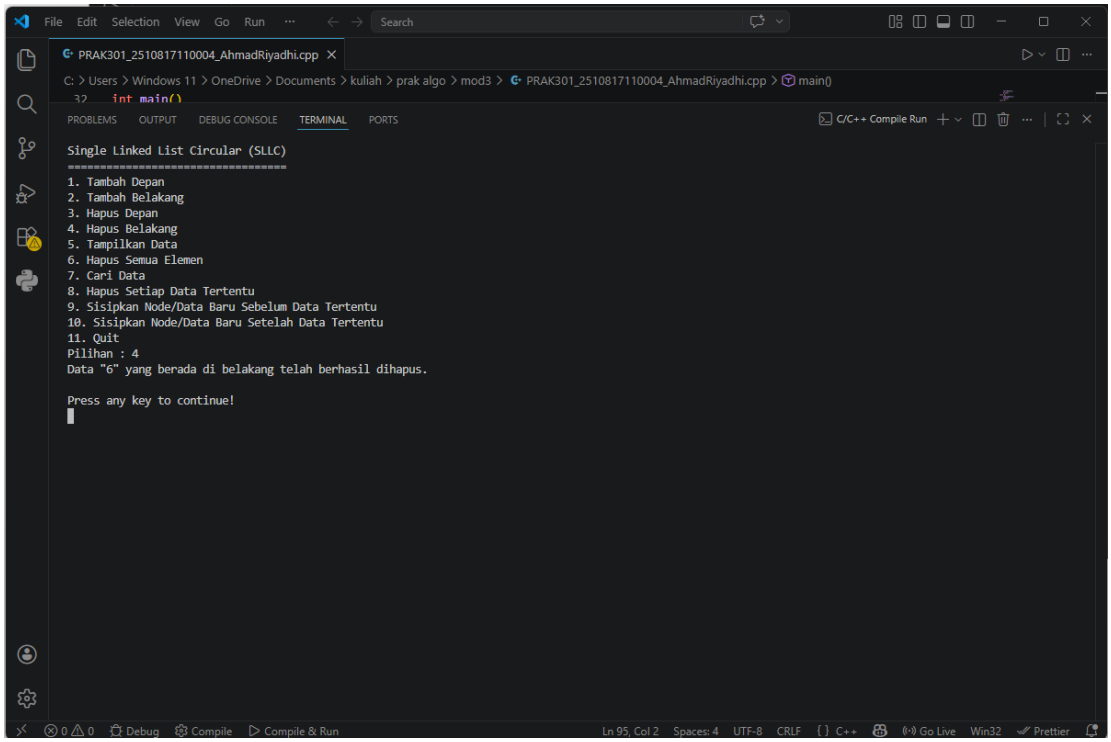
Gambar 5 Tampilan Data Setelah Dilakukan Tambah Data Belakang



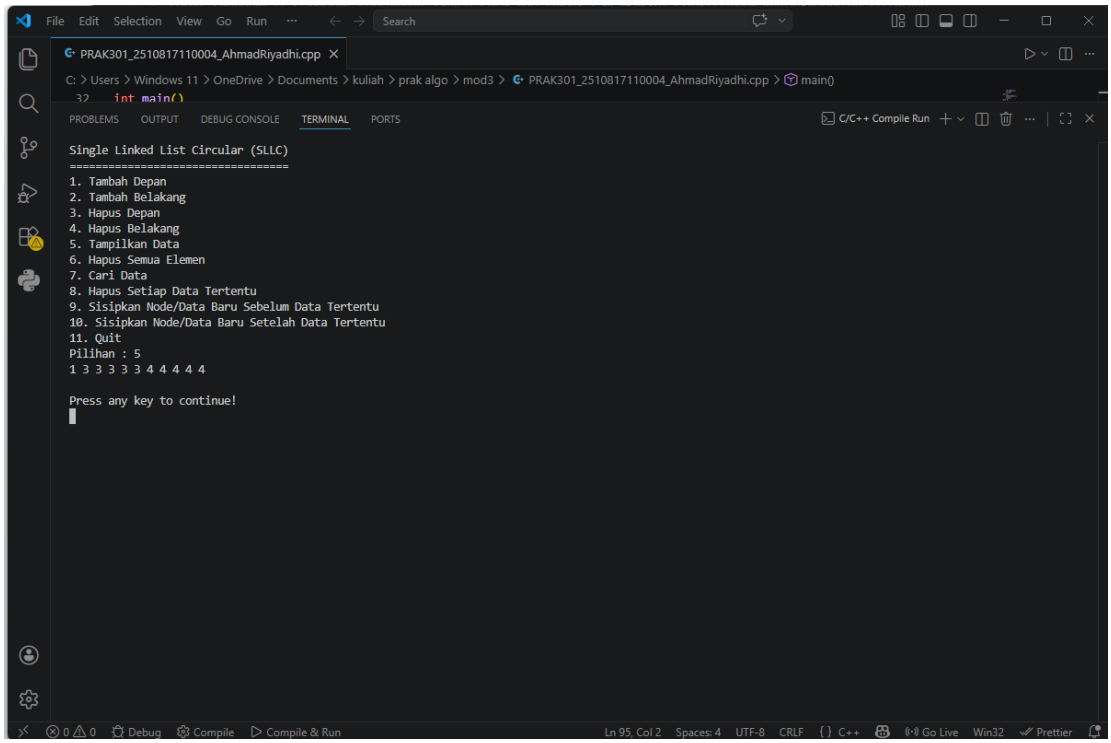
Gambar 6 Tampilan Cari Data Yang Tidak Ada



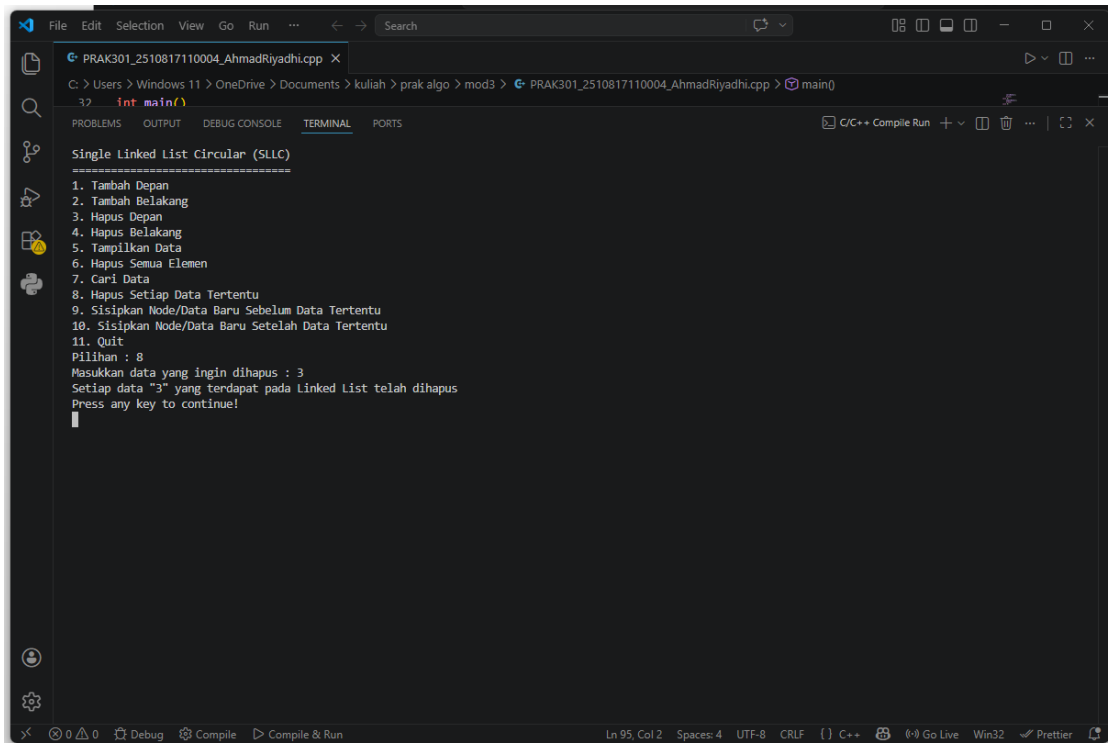
Gambar 7 Tampilan Cari Data Yang Ada



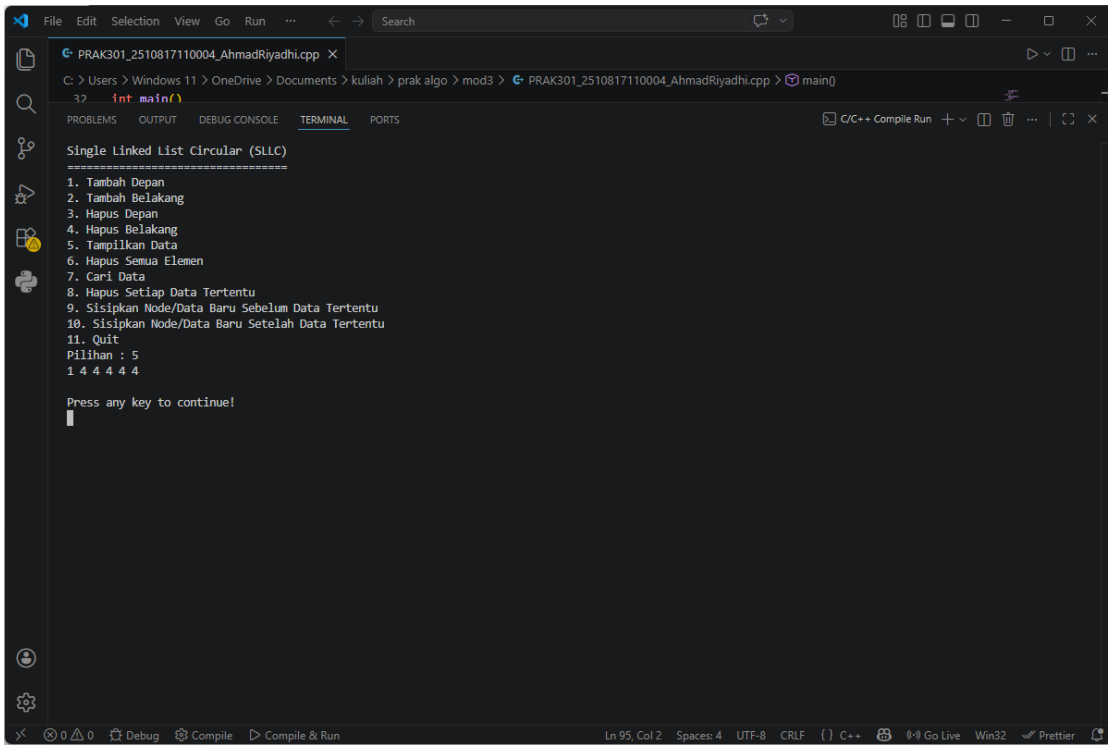
Gambar 8 Tampilan Hapus Data Belakang



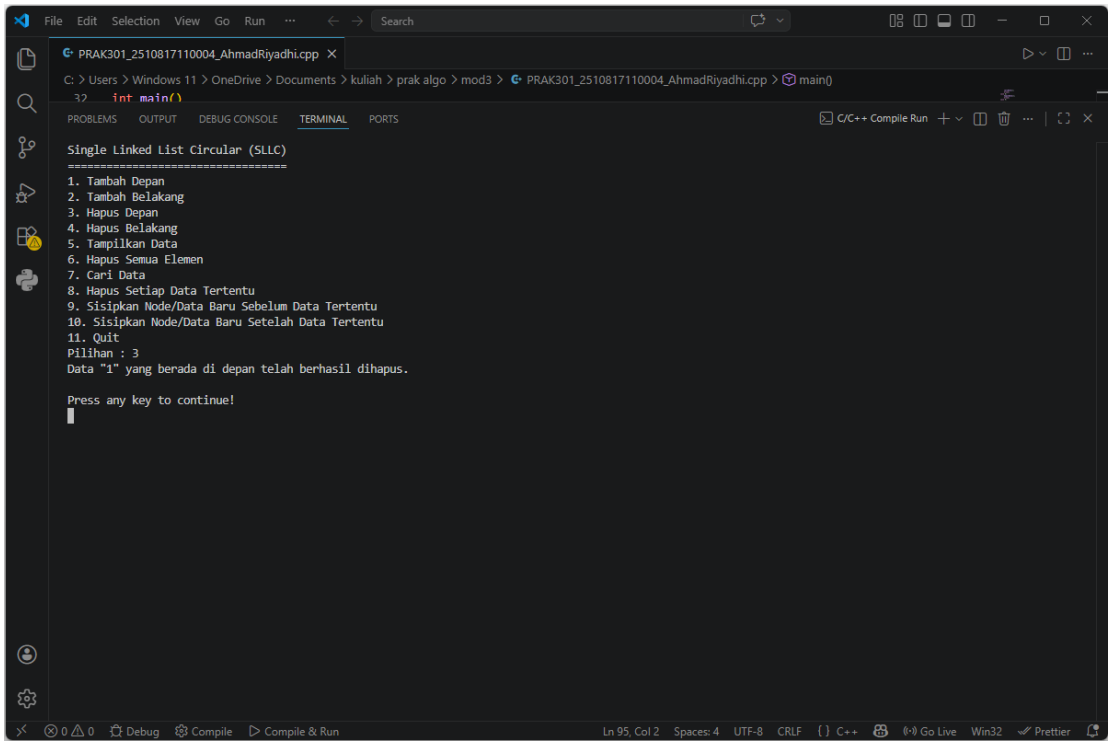
Gambar 9 Tampilan Data Setelah Dilakukan Hapus Data Belakang



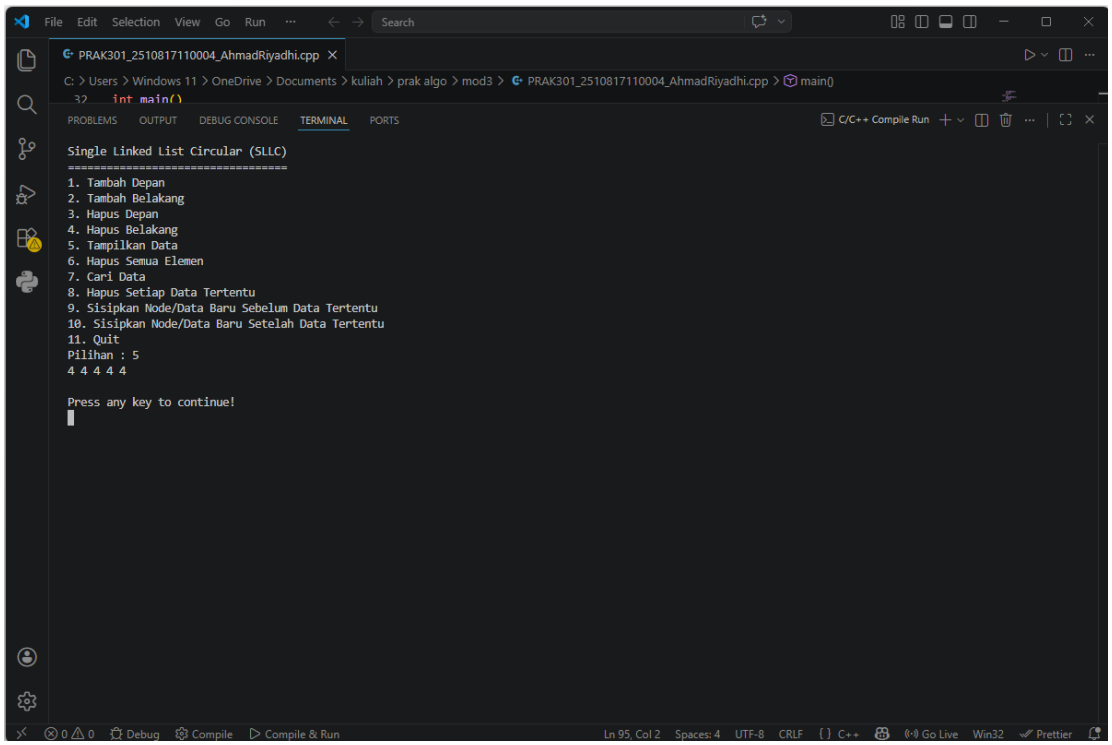
Gambar 10 Tampilan Hapus Setiap Data Tertentu



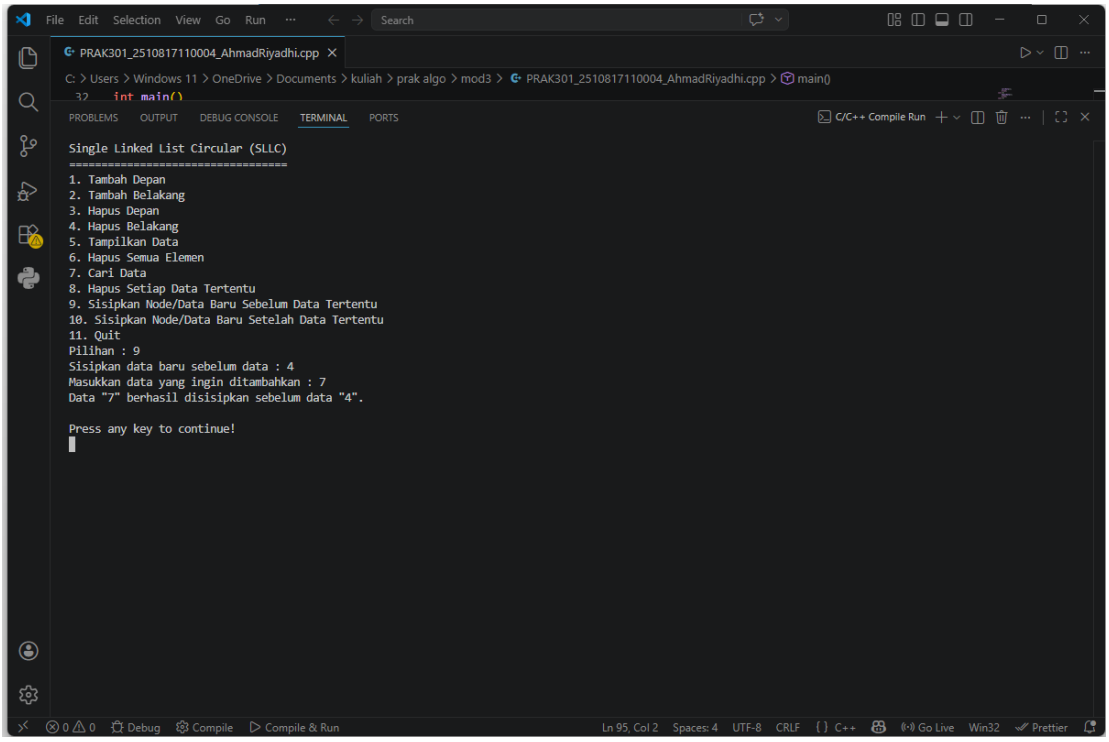
Gambar 11 Tampilan Data Setelah Dilakukan Hapus Setiap Data Tertentu



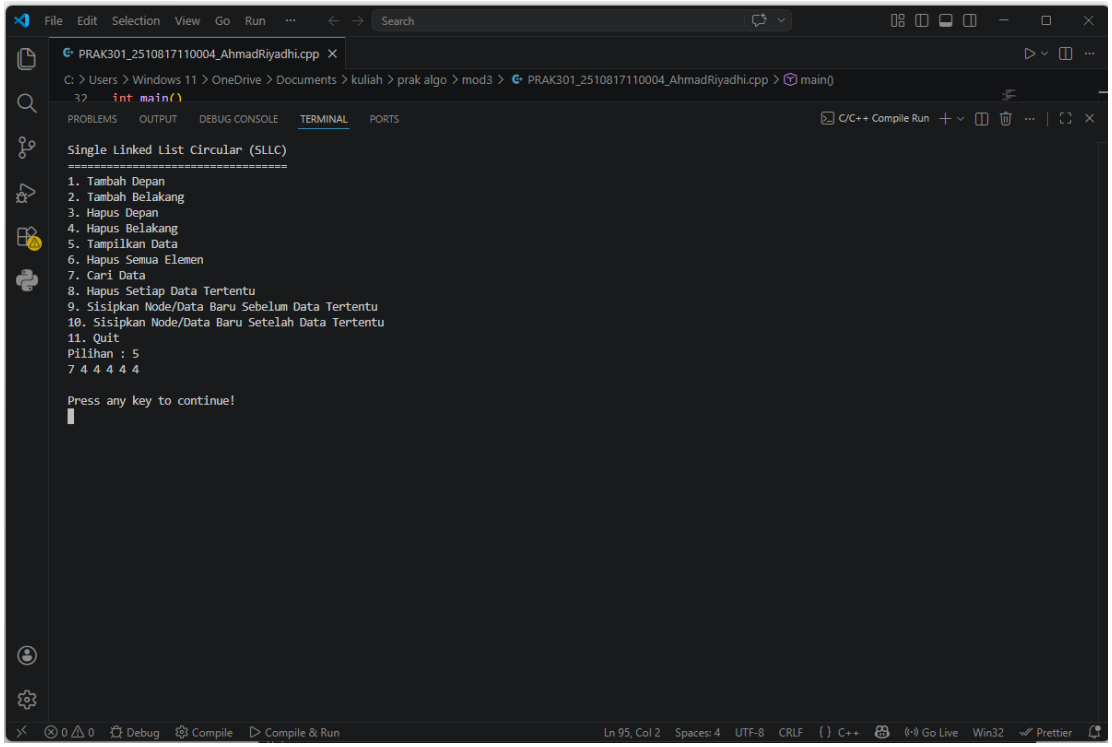
Gambar 12 Tampilan Hapus Data Depan



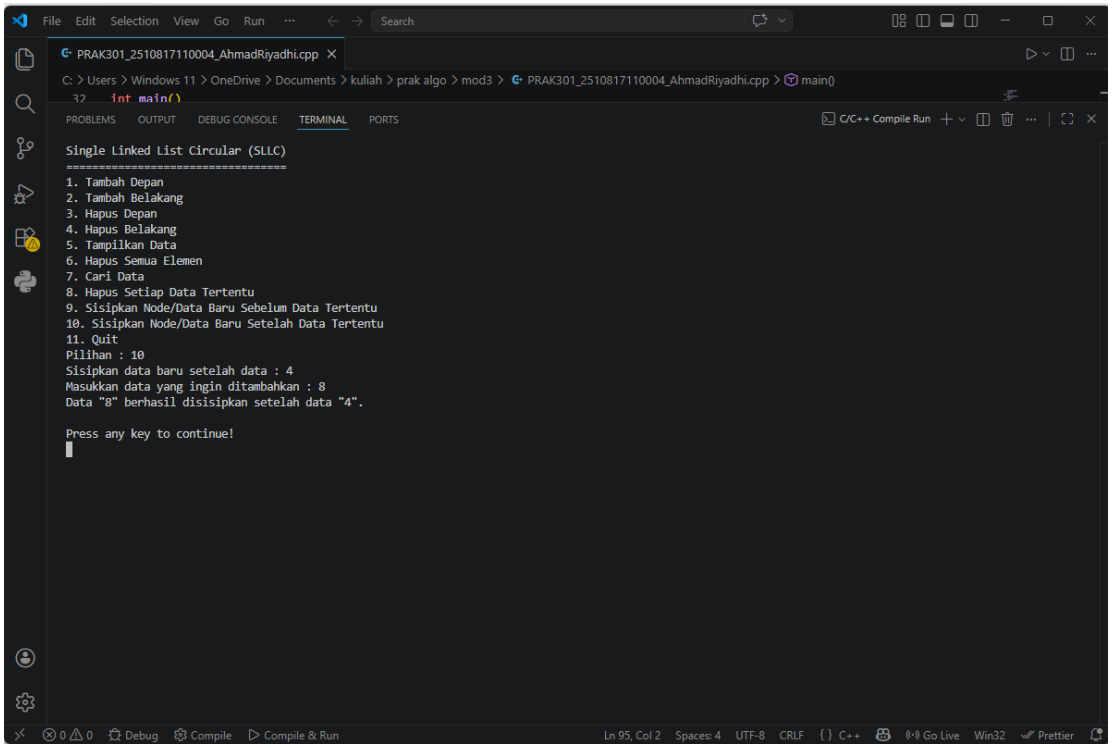
Gambar 13 Tampilan Data Setelah Dilakukan Hapus Data Depan



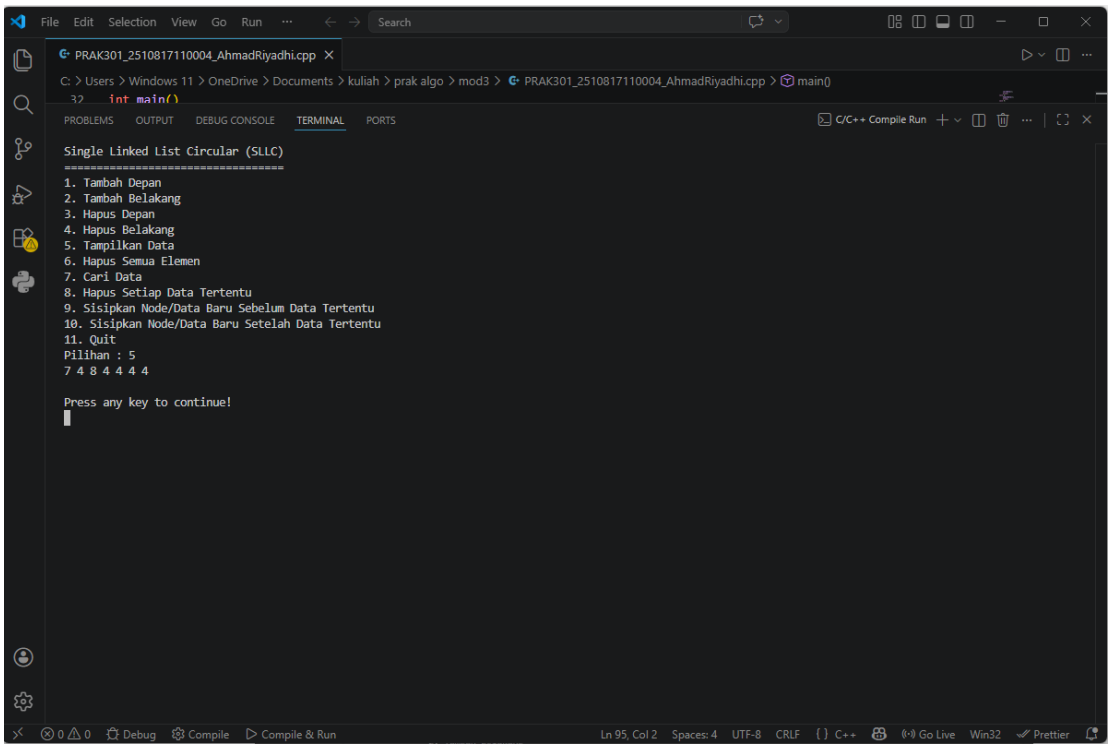
Gambar 14 Tampilan Sisipkan Node Sebelum Data Tertentu



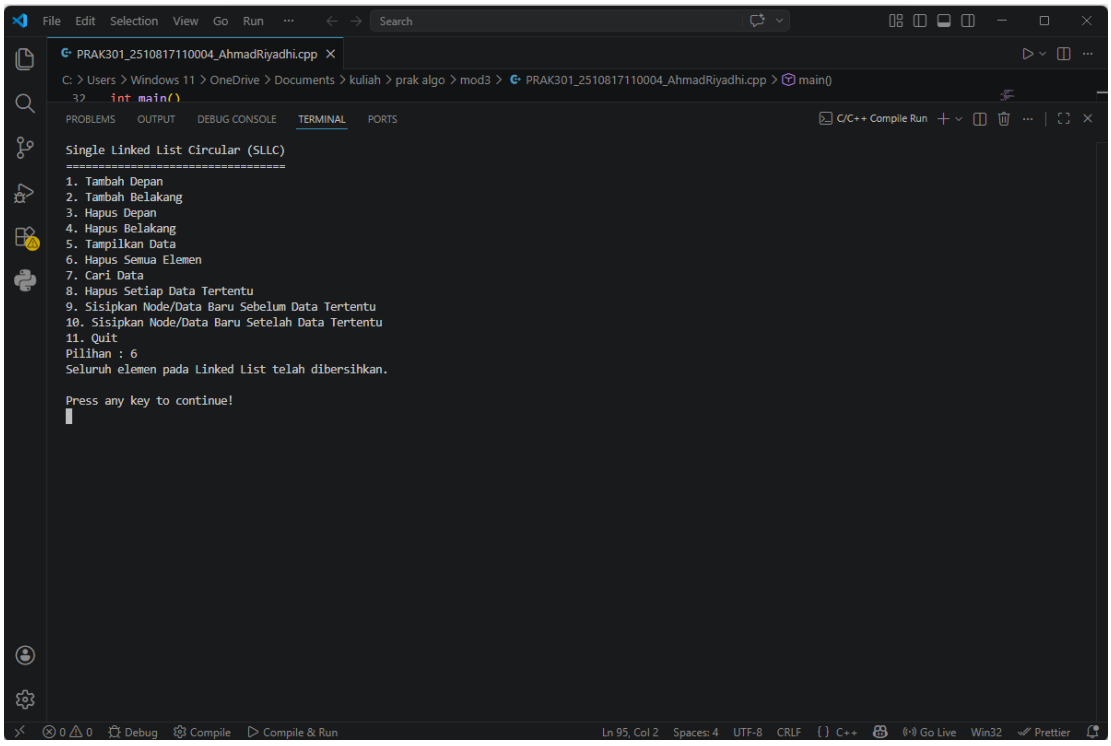
Gambar 15 Tampilan Data Setelah Dilakukan Sisipkan Node Sebelum



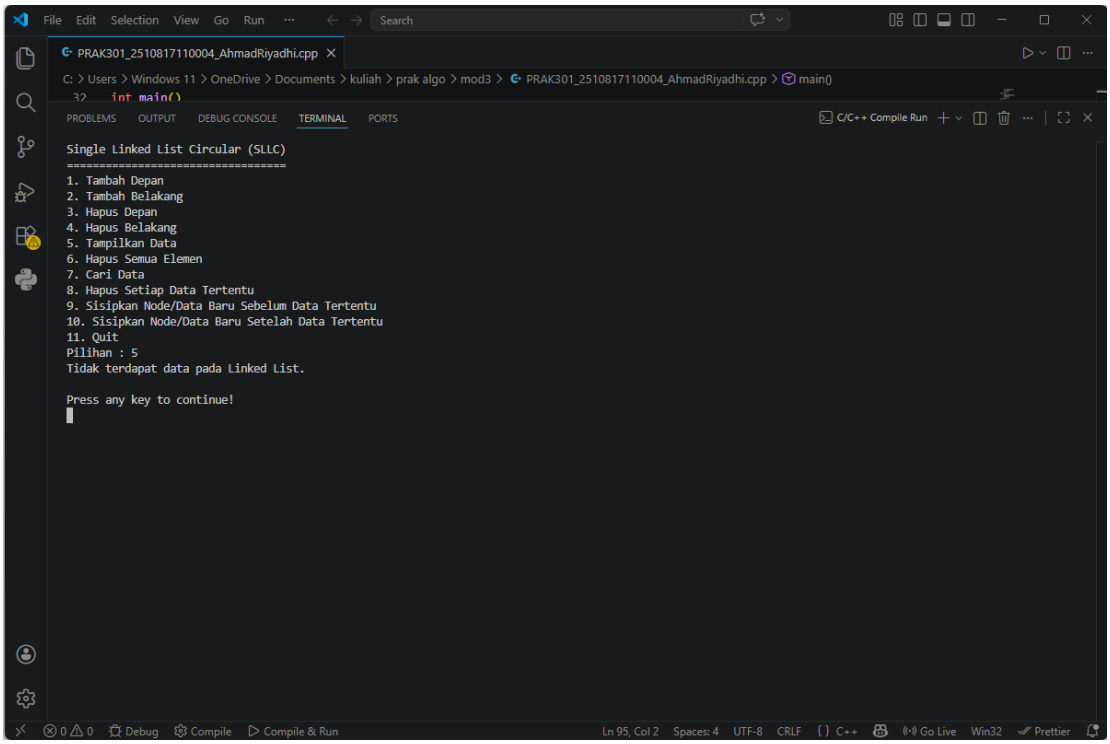
Gambar 16 Tampilan Sisipkan Node Setelah Data Tertentu



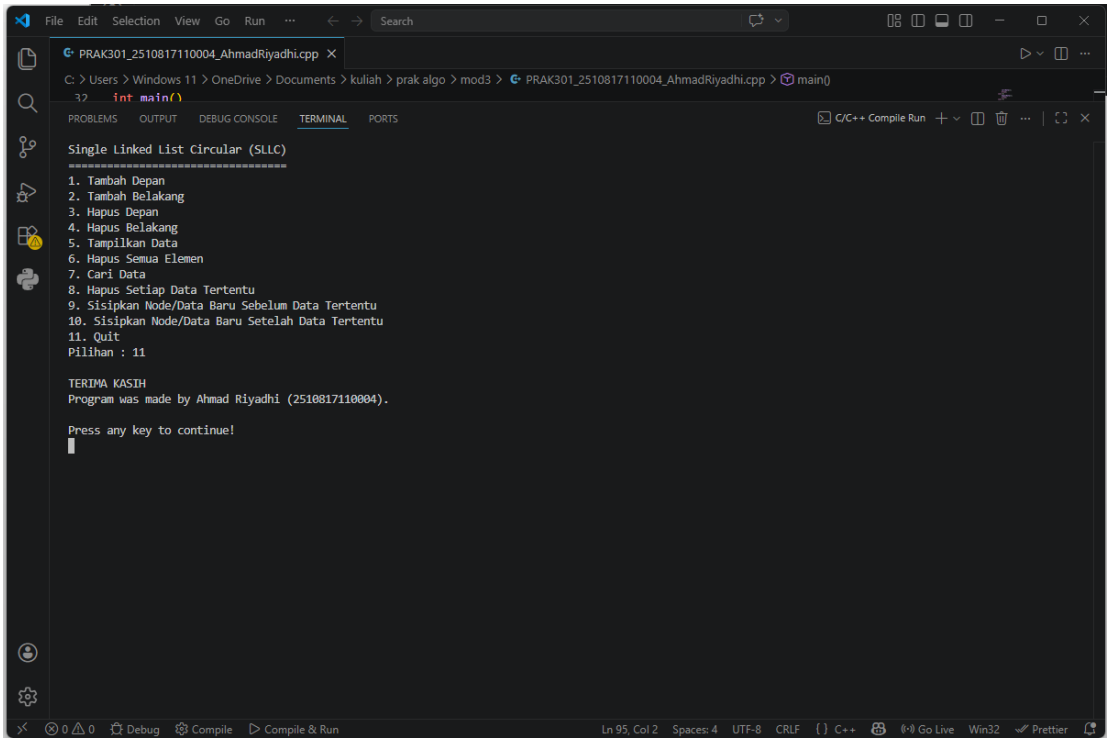
Gambar 17 Tampilan Data Setelah Dilakukan Sisipkan Node Setelah



Gambar 18 Tampilan Hapus Semua Elemen



Gambar 19 Tampilan Data Setelah Dilakukan Hapus Semua Elemen



Gambar 20 Tampilan Setelah Selesai Menyelesaiakan Program

- **baris 1 - 3**, syntax #include <conio.h>, #include <iostream>, dan #include <stdlib.h> berfungsi untuk memanggil library standar bawaan C++ agar program bisa menerima input dan mengeluarkan teks di layar, serta bisa menggunakan perintah khusus untuk membersihkan layar terminal.
- **baris 5**, syntax using namespace std; berfungsi untuk mempersingkat penulisan kode dasar sehingga kita tidak perlu mengetikkan std:: berulang kali di setiap perintah cetak (cout) atau input (cin).
- **baris 7 - 10**, syntax typedef struct TNode berfungsi untuk menciptakan sebuah cetakan atau tipe data kustom yang di dalamnya menyimpan variabel data (untuk menyimpan teks inputan), serta variabel pointer *next (untuk menyimpan arah sambungan ke data berikutnya).
- **baris 12**, syntax TNode *head, *tail; berfungsi untuk mendeklarasikan pointer utama sebagai penunjuk jalan; di mana head akan menempel pada data urutan pertama, dan tail akan menempel pada data urutan paling akhir.
- **baris 14 - 16**, syntax int pil;, char pilihan[2];, dan string dataBaru, dataDelete; berfungsi untuk menyiapkan ruang memori kosong sementara guna menyimpan angka dari menu yang dipilih pengguna, serta menyimpan teks data yang baru akan ditambahkan atau dihapus.
- **baris 18 - 28**, syntax void init();, int isEmpty();, dan seterusnya berfungsi untuk mendaftarkan nama-nama fungsi ke dalam sistem sebelum digunakan (disebut prototipe), agar program utama di bawahnya mengenali bahwa fungsi-fungsi tersebut ada dan tersedia.
- **baris 30 - 80**, function main() terdapat syntax do { ... } while (pil < 11); yang berfungsi untuk menampilkan menu program secara berulang-ulang selama pengguna tidak mengetik angka 11, kemudian dilanjutkan dengan syntax switch(pil) yang membuat alur program terbagi-bagi untuk menjalankan fungsi spesifik sesuai dengan angka pesanan pengguna, dan diakhiri dengan

syntax system("cls"); yang membuat layar terminal dibersihkan kembali setelah sebuah tugas selesai.

- **baris 82 - 85**, function init() terdapat syntax head = NULL; dan tail = NULL; yang berfungsi untuk memutus semua akses ke dalam urutan data, kemudian dilanjutkan dengan instruksi ini yang membuat keseluruhan Linked List dikembalikan ke kondisi awal atau kosong total.
- **baris 87 - 90**, function isEmpty() terdapat syntax if(head == NULL) yang berfungsi untuk memeriksa apakah antrean memiliki isi atau tidak, kemudian dilanjutkan dengan syntax return 1; atau return 0; yang membuat fungsi tersebut mengirimkan sinyal "kosong" atau "berisi" kepada fungsi-fungsi lain yang membutuhkannya.
- **baris 92 - 100**, function tambahDepan() terdapat syntax cout << "Masukkan data : "; hingga baru->next = baru; yang berfungsi untuk mencetak perintah input ke layar, menyiapkan ruang memori baru melalui new TNode, serta menyambungkan pointer next dari node baru tersebut ke dirinya sendiri sebagai persiapan awal untuk membentuk rantai *circular*.
- **baris 102 - 104**, function isEmpty() dengan syntax if(head == NULL) berfungsi untuk memeriksa apakah antrean memiliki isi atau tidak, kemudian dilanjutkan dengan syntax return 1; atau return 0; yang membuat fungsi tersebut mengirimkan sinyal "kosong" atau "berisi" kepada fungsi-fungsi lain yang membutuhkannya.
- **baris 106 - 121**, function tambahDepan() terdapat syntax baru = new TNode; yang berfungsi untuk mencetak ruang memori untuk data yang baru, kemudian dilanjutkan dengan syntax baru->next = head; dan head = baru; yang membuat data baru tersebut didorong paksa menjadi urutan paling awal, lalu ditutup dengan syntax tail->next = head; yang membuat ujung data paling belakang melemparkan sambungannya kembali ke data pertama tersebut.
- **baris 123 - 138**, function tambahBelakang() terdapat syntax baru = new TNode; yang berfungsi untuk membuat ruang memori baru, kemudian dilanjutkan dengan syntax tail->next = baru; dan tail = baru; yang membuat

data baru tersebut diikat pada bagian paling buntut dari lintasan, lalu diakhiri dengan syntax tail->next = head; yang membuat data paling belakang ini menutup kembali jalur lingkarannya ke arah depan.

- **baris 140 - 155**, function hapusDepan() terdapat syntax hapus = head; yang berfungsi untuk menandai data pertama yang akan dibuang, kemudian dilanjutkan dengan syntax head = head->next; yang membuat label awal antrean dipindah ke urutan kedua, lalu dirapikan dengan syntax tail->next = head;, dan ditutup dengan syntax delete hapus; yang membuat data lama tersebut benar-benar musnah dari memori komputer (RAM).
- **baris 157 - 175**, function hapusBelakang() terdapat syntax while(newTail->next != tail) yang berfungsi untuk memaksa program menelusuri data dari awal demi menemukan elemen tepat sebelum ekor terakhir, kemudian dilanjutkan dengan syntax tail = newTail; yang membuat elemen sebelum ekor tersebut diangkat menjadi ekor yang baru, lalu dipungkasi dengan syntax delete hapus; yang membuat data ekor yang lama dihancurkan.
- **baris 177 - 188**, function tampilkan() terdapat syntax do { ... } while(bantu != head); yang berfungsi untuk menyisir lintasan data terus-menerus hingga kembali ke awal, kemudian dilanjutkan dengan syntax cout << bantu->data; yang membuat nilai dari tiap-tiap data dicetak ke layar terminal sebelum akhirnya pointer berpindah ke data sebelahnya.
- **baris 190 - 203**, function reset() terdapat syntax perulangan yang cara kerjanya sama dengan fitur tampilkan, namun di dalamnya terdapat syntax delete hapus; yang berfungsi untuk menghancurkan fisik memori setiap data yang dilewatinya, kemudian dilanjutkan dengan syntax init(); yang membuat antrean kembali dipastikan kosong.
- **baris 205 - 248**, function cariData() terdapat syntax if(cari == bantu->data) yang berfungsi untuk mendeteksi apakah ketikan pencarian dari pengguna cocok dengan salah satu data, kemudian dilanjutkan dengan syntax cetak bertanda [...] yang membuat data sasaran tersebut diekspos atau ditonjolkan saat susunan data dicetak di layar.

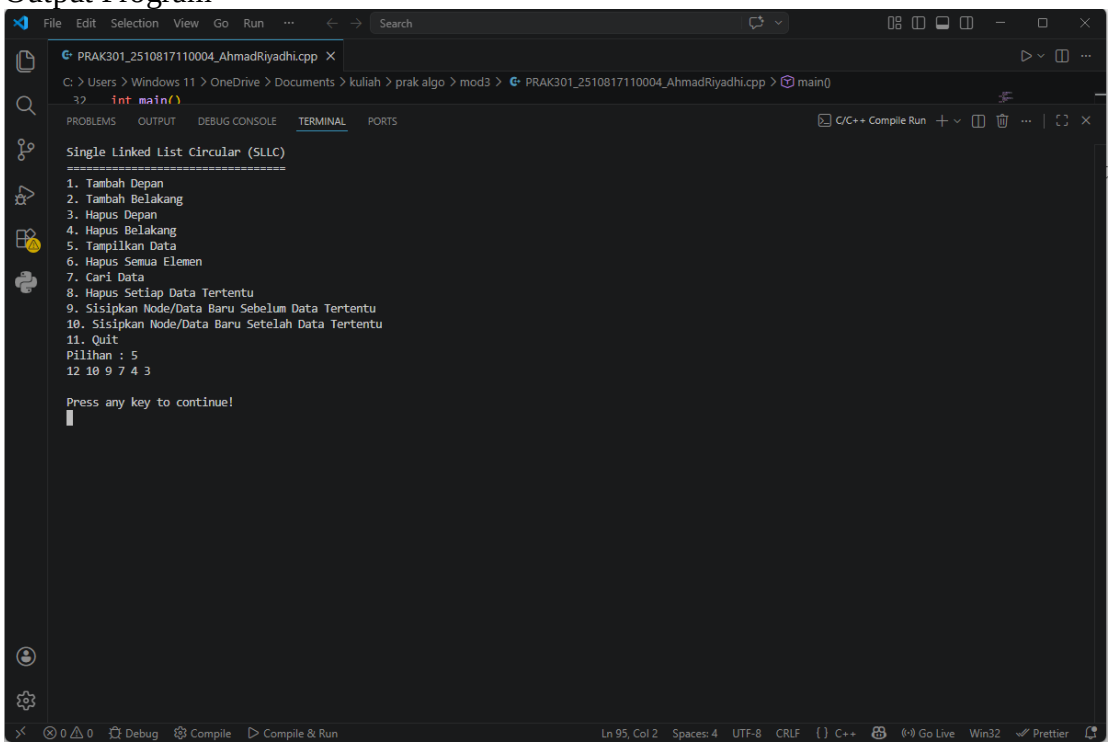
- **baris 250 - 285**, function hapusData() terdapat syntax hapus[hitung++] = bantu; yang berfungsi untuk mengumpulkan semua alamat node yang datanya cocok (sama dengan input cari) ke dalam array hapus[], kemudian dilanjutkan dengan syntax sebelum->next = bantu->next; yang memutus tali rantai agar node yang ingin dihapus "terputus" dari jalur utama tanpa merusak sirkular list.
- **baris 286 - 300**, lanjutan hapusData() terdapat syntax for(int i = 0; i < hitung; i++) yang berfungsi melakukan perulangan untuk menghancurkan node yang sudah ditandai. Di dalamnya terdapat pengecekan if(hapus[i] == head) atau tail untuk memanggil fungsi hapus khusus agar posisi head dan tail tetap akurat setelah penghapusan, lalu ditutup dengan delete hapus[i] untuk membersihkan memori secara total.
- **baris 302 - 340**, function sisipkanSebelum() terdapat syntax do { ... } while(bantu != head); yang berfungsi untuk menelusuri list guna mencari node dengan data target (nextData). Jika ditemukan, apaAda diubah menjadi true. Kemudian dilanjutkan dengan syntax baru->next = bantu; dan sebelum->next = baru; yang menjahit node baru tepat di depan node target tanpa memutuskan sambungan sirkularnya.
- **baris 342 - 379**, function sisipkanSetelah() terdapat syntax prevData == bantu->data yang berfungsi mencari lokasi data patokan. Jika ditemukan, node baru disiapkan dan disambungkan menggunakan syntax baru->next = bantu->next; serta bantu->next = baru;. Ini membuat node baru masuk ke dalam antrean tepat di belakang data patokan, dan jika penyisipan terjadi di elemen paling akhir, syntax if(bantu == tail) akan memastikan tail diperbarui ke node baru tersebut.

SOAL 2

Lakukan tambah data depan 3, 4, 7, 9, 10, 12 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Pembahasan

Output Program



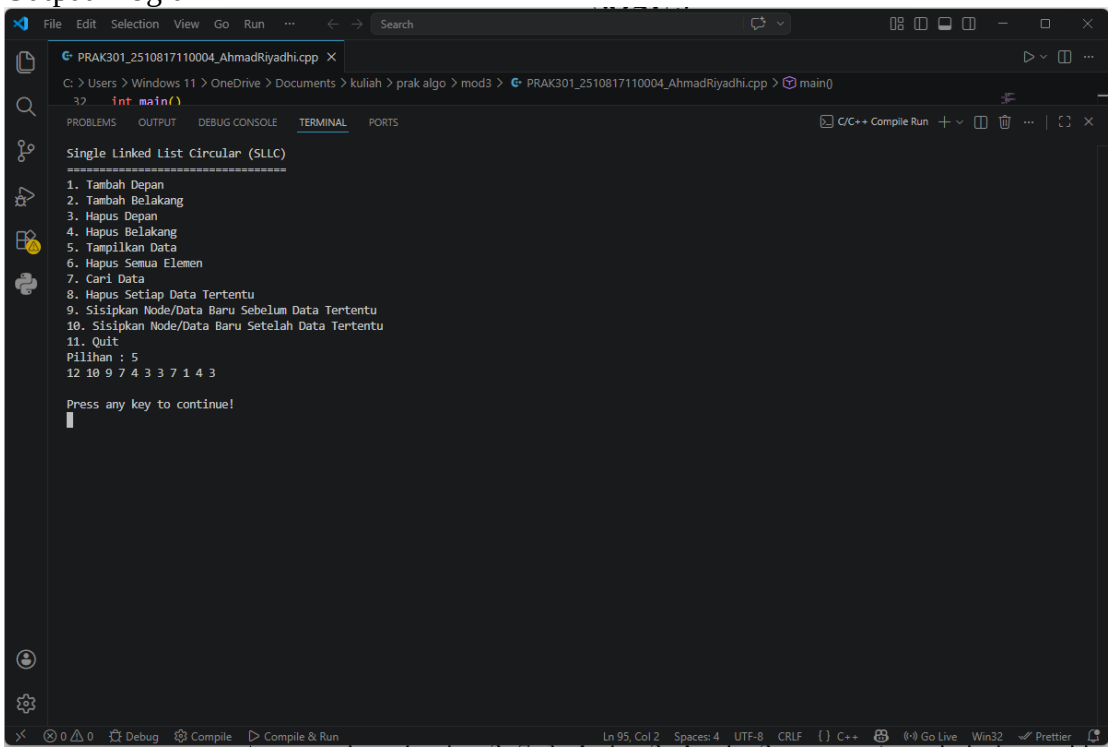
Gambar 21 Tampilan Nilai yang Diinput dari Depan

SOAL 3

Lakukan tambah data belakang 3, 7, 1, 4, 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Pembahasan

Output Program



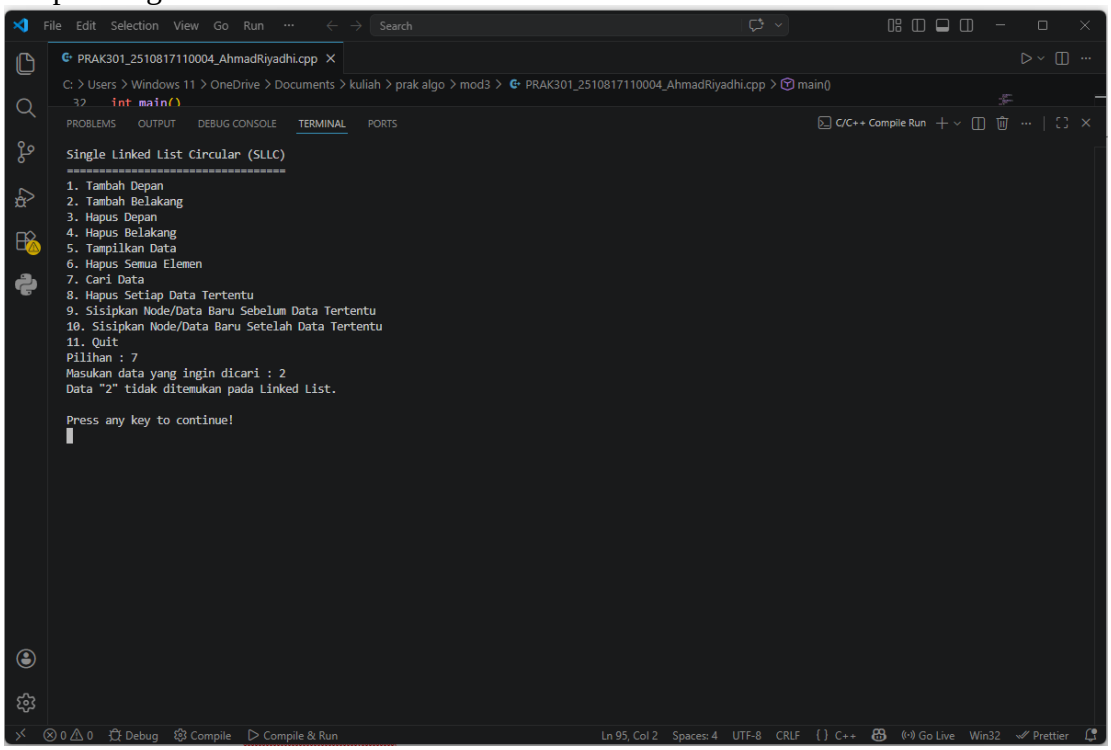
Gambar 22 Tampilan Nilai yang Diinput dari Belakang

SOAL 4

Apa yang terjadi jika mencari angka 2 pada Single Linked List Circular (SLLC) pada data yang telah ditambahkan/dimasukkan sebelumnya dan screenshoot hasilnya

Pembahasan

Output Program



Gambar 23 Pencarian Nilai 2 pada Linked List

Jika melakukan pencarian angka "2" pada SLLC, program menjalankan fungsi cariData() yang melakukan penelusuran (*traversal*) pada seluruh node dalam list.

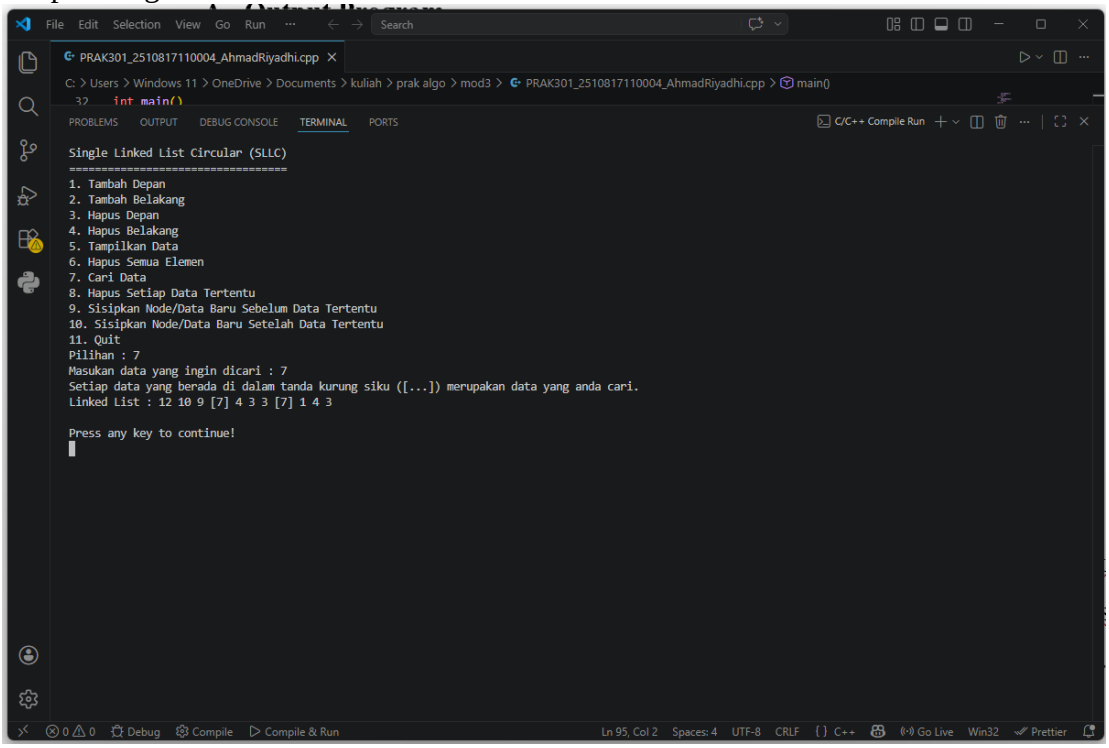
- Program memulai penelusuran dari head dan menggunakan perulangan do-while untuk menyisir setiap node hingga kembali ke titik awal.
- Di setiap node, program membandingkan nilai input dengan data yang tersimpan (if(cari == bantu->data)).
- Jika angka "2" tidak ditemukan setelah seluruh node diperiksa, variabel apaDitemukan tetap bernilai false.
- Sebagai hasil akhir, program mencetak pesan yang menyatakan bahwa data tidak ditemukan pada *Linked List*.

SOAL 5

Coba cari angka 7 dan screenshoot hasilnya !

Pembahasan

Output Program



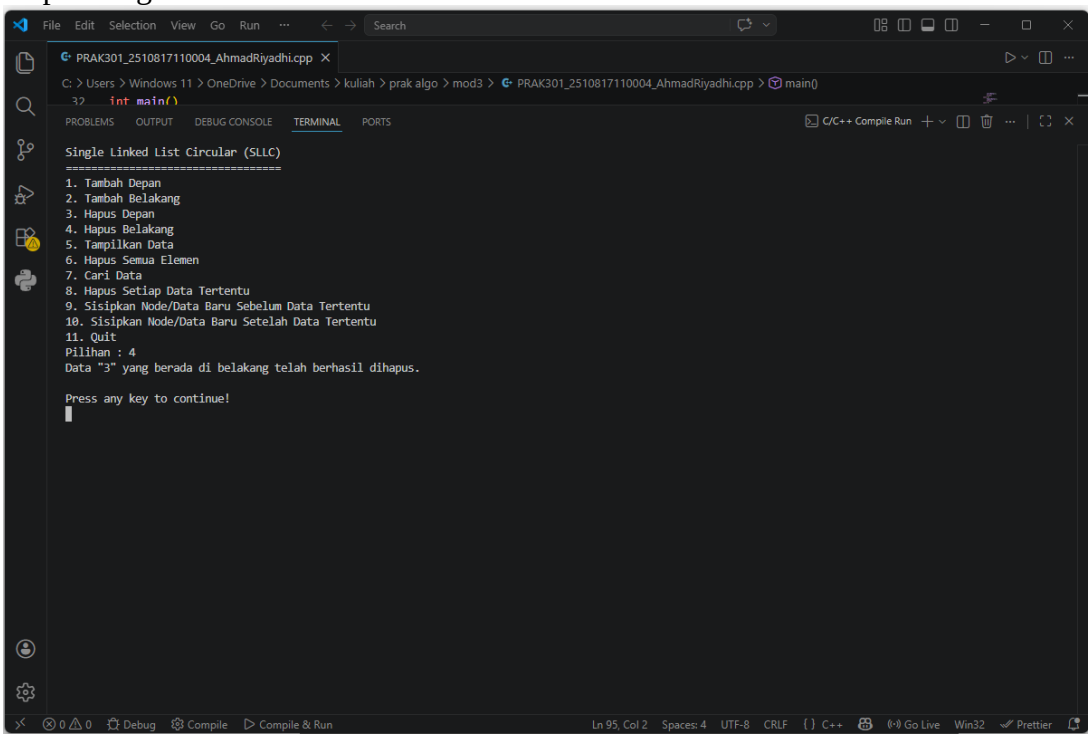
Gambar 24 Pencarian Nilai 7 pada Linked List

SOAL 6

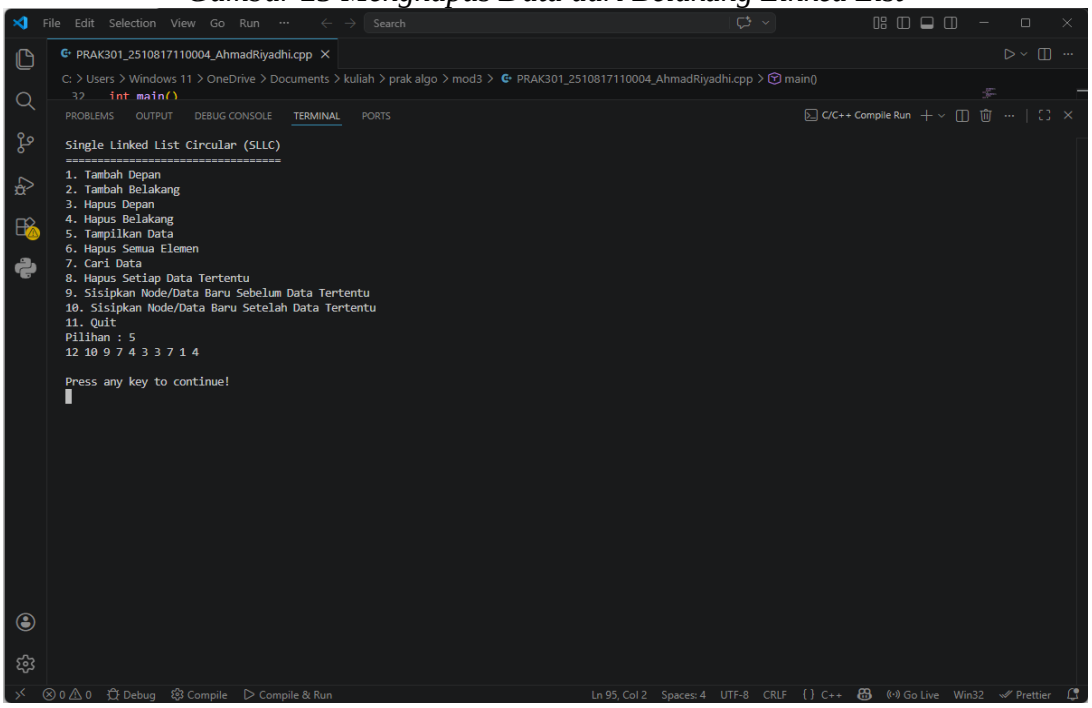
Lakukan hapus belakang dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Pembahasan

Output Program



Gambar 25 Menghapus Data dari Belakang Linked List



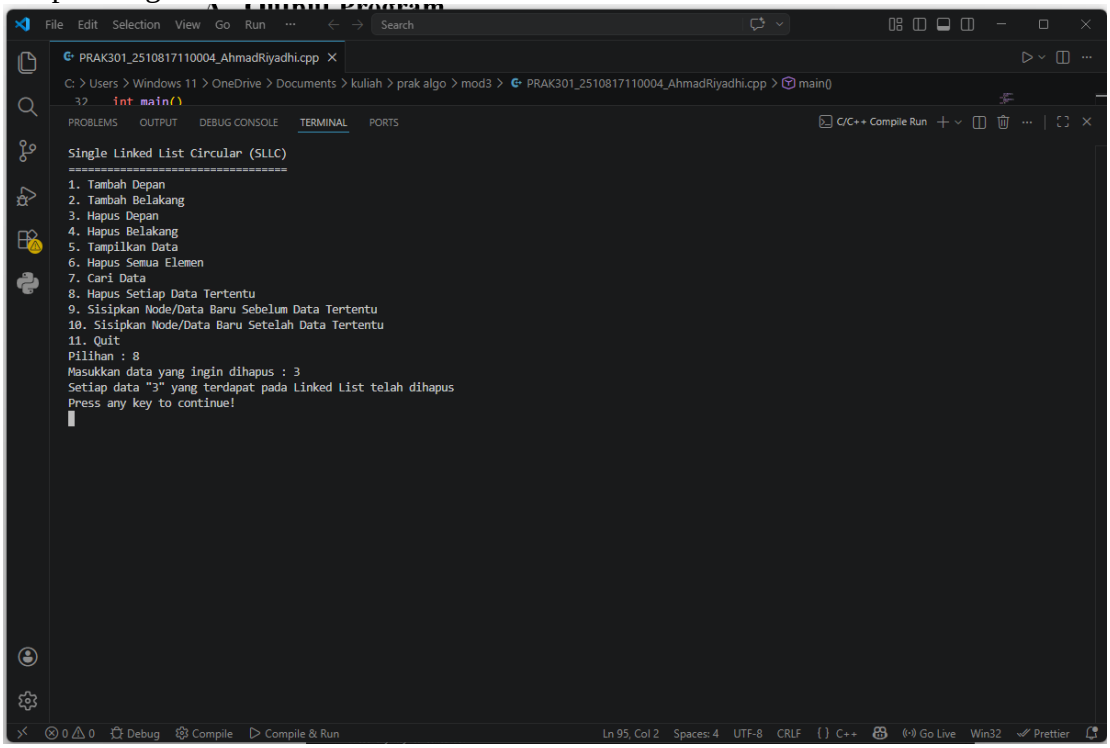
Gambar 26 Tampilan Linked List Setelah Dihapus

SOAL 7

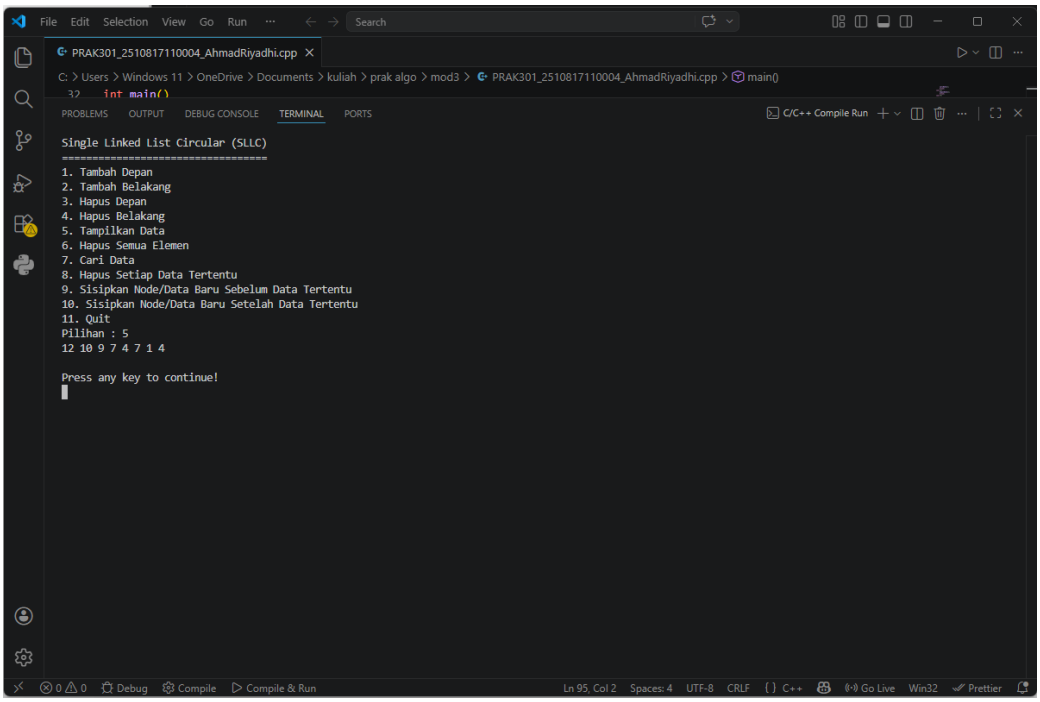
Lakukan hapus setiap angka 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Pembahasan

Output Program



Gambar 27 Menghapus Setiap Nilai 3 yang Ada di Linked List



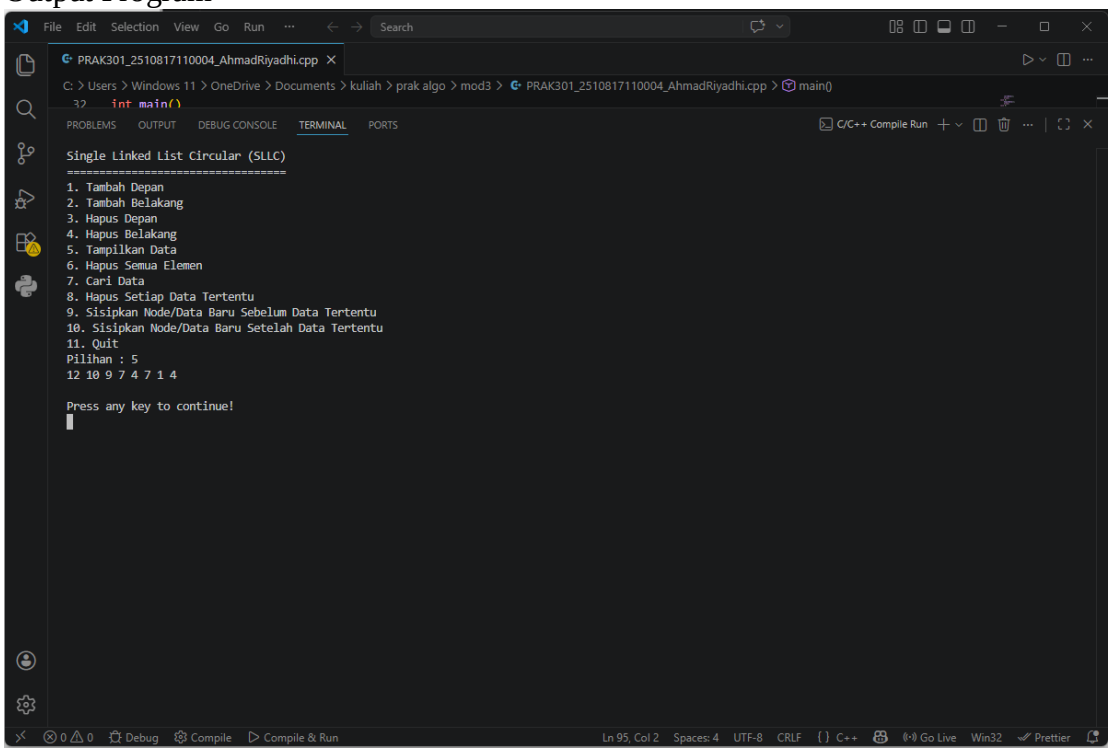
Gambar 28 Tampilan Linked List Setelah Setiap Nilai 3 Dihapus

SOAL 8

Tampilkan data lalu jelaskan yang mana head dan yang mana tail.

Pembahasan

Output Program



Gambar 29 Tampilan Isi dari Linked List

Setelah menjalankan fungsi tampilkan(), program akan melakukan perulangan (*looping*) dari simpul pertama hingga kembali ke posisi semula. Penjelasannya sebagai berikut:

- **Head:** Ini adalah titik awal. Dalam kodingan kamu, head adalah *pointer* yang menyimpan alamat data pertama yang dimasukkan ke dalam list. Saat fungsi tampilkan() berjalan, bantu = head digunakan sebagai titik berangkat pertama kali. Secara visual, head adalah data yang paling pertama muncul di layar saat daftar data dicetak.
- **Tail:** Ini adalah simpul terakhir sebelum lingkaran menutup. Di SLLC, tail bukan hanya data terakhir, tetapi juga pemegang kunci sirkular. Simpul yang kamu sebut tail adalah satu-satunya data yang memiliki *pointer* next menunjuk kembali ke head. Jika kamu melihat output di layar, tail adalah data yang dicetak

tepat sebelum program mendeteksi bahwa perulangan sudah mencapai alamat head kembali.

SOAL 9

Jika baris ke 103 dan 104 dihapus maka apa yang akan terjadi pada saat memasukkan data, dan jelaskan mengapa?

Pembahasan

Baris 103 dan 104 dalam kode kamu merupakan bagian dari instruksi switch(pil) di fungsi main(), yang menangani alur navigasi menu.

- **Dampak:** Fitur "Sisipkan Node/Data Baru Setelah Data Tertentu" jadinya tidak akan bisa dipakai sama sekali.
- **Mengapa:** switch itu bekerja seperti saklar. Baris case 10: (baris 103) adalah line yang memberi tahu program bahwa jika pengguna menekan angka "10", maka instruksi di bawahnya harus dijalankan. Baris sisipkanSetelah(); (baris 104) adalah instruksi untuk menjalankan fungsi penyisipan tersebut. Jika dua baris ini dihapus, program tidak memiliki line untuk menjalankan fungsi tersebut. Meskipun teks menu "10" masih terlihat di layar (karena hanya teks saja tanpa fungsi), program akan mengabaikan input angka 10 karena tidak ada definisi perintah yang harus dilakukan untuk angka tersebut.

SOAL 10

Jelaskan apa itu variabel head dan tail pada SLLC!

Pembahasan

Dalam implementasi struktur data SLLC, head dan tail merupakan variabel bertipe pointer yang berfungsi sebagai pengelola navigasi dan konektivitas node

- Variabel Head

head berfungsi sebagai titik acuan atau alamat memori dari simpul pertama dalam daftar. Variabel ini bertindak sebagai *entry point* atau gerbang utama dalam SLLC. Semua operasi yang memerlukan akses ke awal daftar, seperti melakukan penelusuran data (*traversal*) atau menghapus elemen terdepan, selalu merujuk pada alamat yang tersimpan dalam head .

- Variabel Tail

tail berfungsi sebagai penunjuk alamat dari simpul terakhir dalam daftar. Dalam struktur data linear biasa, *pointer next* pada simpul terakhir biasanya bernilai NULL. Namun, dalam SLLC, tail memiliki peran krusial untuk menjaga integritas struktur sirkular dengan mengatur agar *pointer next* miliknya menunjuk kembali ke alamat head. Mekanisme ini memastikan bahwa daftar data tidak berakhir, melainkan membentuk sebuah siklus yang terus berulang.

- Hubungan dalam SLLC

Hubungan antara head dan tail memungkinkan struktur data ini untuk mempertahankan sifat circular-nya secara permanen selama siklus tail->next = head tetap terjaga. Setiap operasi modifikasi data, baik penambahan maupun penghapusan, akan selalu melakukan penyesuaian pada posisi head dan tail untuk menjamin bahwa jalur sirkular tidak terputus.

TAUTAN GIT HUB

https://github.com/TebangLiar/Praktikum-AlgoritmaDanStrukturData-P1/tree/main/Modul%203